Práctica 1 de Modbus TCP: un cliente escrito a mano

Presentación

En esta práctica se escribe en Python, byte a byte, un cliente Modbus TCP completo y se prueba contra el simulador del laboratorio, que hace de servidor, analizando cada trama recibida. Si un campo está mal, el simulador indica cuál, qué esperaba y qué ha hecho con la trama.

La práctica es **individual** y no se proporciona ningún código de partida: el programa se escribe desde cero.

La práctica es **incremental**. Para aprobar basta con el nivel básico; quien quiera optar a la máxima nota implementa además el nivel avanzado.

Nivel	Qué hay que implementar	Nota máxima
Básico (necesario para aprobar)	Cabecera MBAP y cliente; lecturas 01, 02, 03 y 04; escrituras de un solo elemento 05 y 06; tratamiento de las respuestas de excepción	5
Avanzado	Además: escrituras de varios elementos 0F y 10, la función combinada 17 y el programa `modbus_cli.py`	10

Material y puesta en marcha

Se trabaja en un solo PC: el simulador (servidor) y el programa propio (cliente) se comunican por la interfaz local `127.0.0.1`.

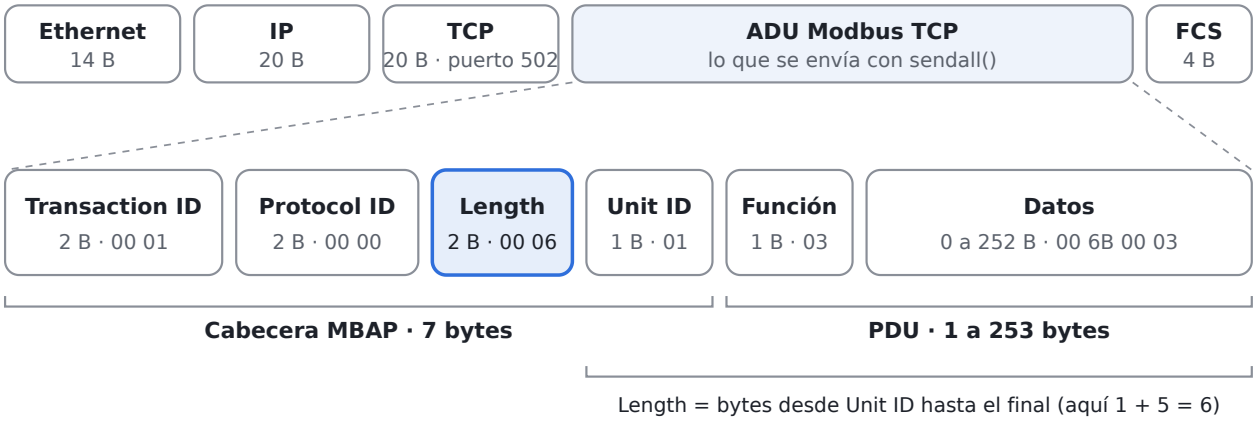
Material	Para qué sirve
`SimuladorModbus` (ejecutable para Linux)	Servidor Modbus TCP con memoria visible y analizador de tramas
Este guion	Documentación del protocolo: todas las funciones y sus tramas

Entrega: una carpeta `practica1` con dos archivos, `modbus_tcp.py` (la biblioteca) y `modbus_cli.py` (el programa de línea de órdenes). Solo se puede usar la biblioteca estándar de Python. No se permiten bibliotecas Modbus.

1. Abrir un terminal en la carpeta del simulador. La primera vez, darle permiso de ejecución (`chmod +x SimuladorModbus`). Después, arrancarlo con `./SimuladorModbus`: se abre el navegador con la interfaz. En esta práctica se usan sobre todo dos pestañas: **Servidor y memoria**, para ver y cambiar los datos del servidor, y **Analizador de tramas**, para ver cada trama recibida analizada campo a campo. Las demás no hacen falta.
2. Anotar el puerto que indica el terminal. En Linux, un usuario sin permisos de administrador no puede abrir puertos por debajo de 1024, así que el simulador no puede usar el 502 y arranca automáticamente en el **5020**. Ese es el puerto que hay que indicar al crear el cliente.
3. Abrir Wireshark sobre la interfaz `lo` (loopback) con el filtro `tcp.port == 5020`. Wireshark solo reconoce Modbus por sí mismo en el puerto 502; para que analice las tramas en el 5020, usar *Analizar* › *Decodificar como...* y asignar el puerto TCP 5020 a *Modbus/TCP*. Si la interfaz no aparece, es que el usuario no tiene permiso de captura: avisar al profesor.
4. Para probar cada parte del código hay dos herramientas, de más rápida a más completa:
   - **Contra el simulador:** ejecutar la petición y comprobar en la pestaña *Servidor y memoria* que el valor ha llegado a la dirección correcta.
   - **Analizador de tramas del simulador:** cada petición aparece analizada campo a campo; si un campo está mal, indica cuál, qué esperaba y qué ha hecho el servidor.

Cómo viaja una petición: encapsulación y cabecera MBAP

Modbus TCP reutiliza la PDU de Modbus serie (código de función + datos) y le antepone una cabecera de 7 bytes, la MBAP. Desaparecen la dirección de esclavo y el CRC: el destino lo da la IP y la integridad la garantizan Ethernet y TCP.



encapsulación de una petición FC03 · MBAP + PDU dentro de TCP/IP

El ejemplo de la figura es la petición de los apuntes (figura 8.2): leer tres registros a partir del 4:0108, ahora sobre TCP. Todos los campos de más de un byte van en **big-endian** (primero el byte más significativo).

Bytes	Campo	Tamaño	Quién lo pone	Contenido
0–1	Transaction ID	2	Cliente; el servidor lo copia	Número de transacción. Permite emparejar respuesta y petición
2–3	Protocol ID	2	Cliente; el servidor lo copia	Siempre 0 (Modbus)
4–5	Length	2	Quien envía la trama	Número de bytes que siguen: 1 (Unit ID) + longitud de la PDU. Máximo 254
6	Unit ID	1	Cliente; el servidor lo copia	Equipo final detrás de una pasarela. En un servidor TCP directo suele ignorarse; valor recomendado 0xFF
7	Código de función	1	Cliente	Qué se pide (ver las secciones siguientes). En la respuesta, el mismo código o el código + 0x80 si hay excepción
8…	Datos	0–252	Cliente / servidor	Depende de la función

Por qué existe Length. TCP entrega un flujo de bytes, no mensajes: una respuesta puede llegar partida en dos `recv()` o junto con la siguiente. Para leer una respuesta completa, el cliente lee primero los 7 bytes de la cabecera y después exactamente `Length − 1` bytes más.

Qué hace el servidor si la cabecera está mal. Si Protocol ID no es 0 o Length es imposible, el servidor no puede saber dónde acaba la trama: la **descarta sin responder** y el cliente ve un *timeout*. Si la cabecera es correcta pero la PDU no, responde con una **excepción**.

Modelo de datos y direccionamiento

Un servidor Modbus expone cuatro tablas. Cada función trabaja sobre una sola tabla, y la dirección que viaja en la trama es la posición dentro de esa tabla.

Tabla	Tipo	Acceso del cliente	Notación clásica	Funciones
Coils (salidas discretas)	1 bit	Lectura y escritura	0:xxxx (0xxxx)	01, 05, 0F
Discrete inputs (entradas discretas)	1 bit	Solo lectura	1:xxxx (1xxxx)	02
Input registers (registros de entrada)	16 bits	Solo lectura	3:xxxx (3xxxx)	04
Holding registers (registros internos)	16 bits	Lectura y escritura	4:xxxx (4xxxx)	03, 06, 10, 17

Direcciones de protocolo, base 0. En la trama, la primera posición de cada tabla es la dirección 0 y la última la 65535. La notación clásica de los fabricantes empieza en 1 y antepone el número de tabla: el registro 4:0108 de los apuntes se pide con la dirección **107** (0x006B) y la función 03. El prefijo nunca viaja en la trama: lo determina la función.

Los registros no tienen tipo. Un registro son 16 bits. Que represente un entero sin signo (0…65535), con signo en complemento a 2 (−32768…32767) o media palabra de un real de 32 bits lo decide el fabricante, no el protocolo. Ejemplo: −5 viaja como `FF FB`.

El simulador. Cada tabla tiene 10 000 posiciones (0–9999). Pedir más allá provoca la excepción 0x02.

Funciones de lectura: 01, 02, 03 y 04

Las cuatro tienen la misma petición (dirección inicial + cantidad) y la misma estructura de respuesta (número de bytes + datos). Solo cambian la tabla y la codificación: bits empaquetados en 01/02, registros de 2 bytes en 03/04. Cada trama se dibuja como una tabla con los campos en el mismo orden en que viajan, de izquierda a derecha. La fila Bytes da el tamaño de cada campo, Valores lo que admite y Ejemplo los bytes reales en hexadecimal con su significado. Debajo aparece la misma trama tal como la imprime la traza de la biblioteca (TX enviada, RX recibida), para compararla directamente.

01 · Read Coils y 02 · Read Discrete Inputs

Ejemplo de las tablas: leer 10 coils desde la dirección 20, que valen 1,0,1,1,0,0,1,0,1,1.

Petición

	Cabecera MBAP	Función	Dirección inicial	Cantidad de bits
Bytes	7	1	2	2
Valores	Length = 6	`01` o `02`	0–65535	1–2000
Ejemplo	`00 01 00 00 00 06 01`	`01`	`00 14` = 20	`00 0A` = 10

Respuesta

	Cabecera MBAP	Función	Número de bytes N	Estado de los bits
Bytes	7	1	1	N
Valores	Length = 3 + N	`01` o `02`	[cantidad / 8]	8 bits por byte
Ejemplo	`00 01 00 00 00 05 01`	`01`	`02`	`4D 03`

```
TX: 00 01 00 00 00 06 01 01 00 14 00 0A
RX: 00 01 00 00 00 05 01 01 02 4D 03
```

Empaquetado de bits. El primer bit pedido va en el bit 0 (el de menos peso) del primer byte; el noveno, en el bit 0 del segundo byte. Los bits sobrantes del último byte se rellenan con 0.

`4D` = `0100 1101`: leyendo de derecha a izquierda, coils 20 a 27 = 1,0,1,1,0,0,1,0. `03` = `0000 0011`: coils 28 y 29 = 1,1, y seis bits de relleno.

Ejemplo FC02: leer 5 entradas discretas desde la 0, que valen 1,1,0,0,1.

```
TX: 00 02 00 00 00 06 01 02 00 00 00 05
RX: 00 02 00 00 00 04 01 02 01 13
```

03 · Read Holding Registers y 04 · Read Input Registers

Ejemplo de las tablas (el de la figura 8.2 de los apuntes, ahora sobre TCP): leer 3 registros desde el 4:0108 (dirección 107), que valen 555, 0 y 100.

Petición

	Cabecera MBAP	Función	Dirección inicial	Cantidad de registros
Bytes	7	1	2	2
Valores	Length = 6	`03` o `04`	0–65535	1–125
Ejemplo	`00 03 00 00 00 06 01`	`03`	`00 6B` = 107	`00 03` = 3

Respuesta

	Cabecera MBAP	Función	Número de bytes N	Valores de los registros
Bytes	7	1	1	N
Valores	Length = 3 + N	`03` o `04`	2 × cantidad	2 bytes por registro, primero el más significativo
Ejemplo	`00 03 00 00 00 09 01`	`03`	`06`	`02 2B 00 00 00 64` = 555, 0, 100

```
TX: 00 03 00 00 00 06 01 03 00 6B 00 03
RX: 00 03 00 00 00 09 01 03 06 02 2B 00 00 00 64
```

Ejemplo FC04: leer 1 registro de entrada desde la dirección 8, que vale 10.

```
TX: 00 04 00 00 00 06 01 04 00 08 00 01
RX: 00 04 00 00 00 05 01 04 02 00 0A
```

Por qué 125 registros y 2000 bits. La PDU de respuesta no puede pasar de 253 bytes: 2 bytes de cabecera + 250 de datos = 125 registros. Con bits, 2000 / 8 = 250 bytes.

Funciones de escritura: 05, 06, 0F y 10

Las escrituras simples (05, 06) responden con un **eco exacto** de la petición. Las múltiples (0F, 10) responden confirmando solo la dirección y la cantidad escritas.

05 · Write Single Coil

Ejemplo de la tabla: poner a ON la coil 172.

Petición

	Cabecera MBAP	Función	Dirección de la coil	Valor
Bytes	7	1	2	2
Valores	Length = 6	`05`	0–65535	`FF 00` = ON, `00 00` = OFF. Cualquier otro provoca la excepción 0x03
Ejemplo	`00 05 00 00 00 06 01`	`05`	`00 AC` = 172	`FF 00` = ON

Respuesta: idéntica a la petición (eco).

```
TX: 00 05 00 00 00 06 01 05 00 AC FF 00
RX: 00 05 00 00 00 06 01 05 00 AC FF 00
```

06 · Write Single Register

Ejemplo de la tabla: escribir 3 en el registro 1.

Petición

	Cabecera MBAP	Función	Dirección del registro	Valor
Bytes	7	1	2	2
Valores	Length = 6	`06`	0–65535	0–65535 (o −32768…32767 en complemento a 2)
Ejemplo	`00 06 00 00 00 06 01`	`06`	`00 01` = 1	`00 03` = 3

Respuesta: idéntica a la petición (eco).

```
TX: 00 06 00 00 00 06 01 06 00 01 00 03
RX: 00 06 00 00 00 06 01 06 00 01 00 03
```

0F (15) · Write Multiple Coils

Ejemplo de las tablas: escribir 10 coils desde la 19 con los valores 1,0,1,1,0,0,1,1,1,0.

Petición

	Cabecera MBAP	Función	Dirección inicial	Cantidad de coils	Número de bytes N	Valores
Bytes	7	1	2	2	1	N
Valores	Length = 7 + N	`0F`	0–65535	1–1968	⌈cantidad / 8⌉	Bits empaquetados como en la 01
Ejemplo	`00 07 00 00 00 09 01`	`0F`	`00 13` = 19	`00 0A` = 10	`02`	`CD 01`

Respuesta

	Cabecera MBAP	Función	Dirección inicial	Cantidad de coils
Bytes	7	1	2	2
Valores	Length = 6	`0F`	La de la petición	La de la petición
Ejemplo	`00 07 00 00 00 06 01`	`0F`	`00 13`	`00 0A`

```
TX: 00 07 00 00 00 09 01 0F 00 13 00 0A 02 CD 01
RX: 00 07 00 00 00 06 01 0F 00 13 00 0A
```

`CD` = `1100 1101` (coils 19 a 26 = 1,0,1,1,0,0,1,1) y `01` = `0000 0001` (coils 27 y 28 = 1,0).

10 (16) · Write Multiple Registers

Ejemplo de las tablas: escribir 0x000A y 0x0102 en los registros 1 y 2.

Petición

	Cabecera MBAP	Función	Dirección inicial	Cantidad de registros	Número de bytes N	Valores
Bytes	7	1	2	2	1	N
Valores	Length = 7 + N	`10`	0–65535	1–123	2 × cantidad	2 bytes por registro, big-endian
Ejemplo	`00 08 00 00 00 0B 01`	`10`	`00 01` = 1	`00 02` = 2	`04`	`00 0A 01 02`

Respuesta

	Cabecera MBAP	Función	Dirección inicial	Cantidad de registros
Bytes	7	1	2	2
Valores	Length = 6	`10`	La de la petición	La de la petición
Ejemplo	`00 08 00 00 00 06 01`	`10`	`00 01`	`00 02`

```
TX: 00 08 00 00 00 0B 01 10 00 01 00 02 04 00 0A 01 02
RX: 00 08 00 00 00 06 01 10 00 01 00 02
```

Por qué 123 registros y 1968 coils. La PDU de petición tiene 6 bytes fijos y no puede pasar de 253: quedan 246 bytes de datos, es decir, 123 registros o 246 × 8 = 1968 bits.

Función combinada: 17

Resuelve en una sola transacción lo que de otro modo exigiría dos (escribir y leer), y evita que otro cliente modifique los registros entre la escritura y la lectura.

17 (23) · Read/Write Multiple Registers

Escribe un bloque de holding registers y lee otro en la misma transacción. La norma fija el orden: **primero se escribe, después se lee**.

Ejemplo de las tablas (el de los apuntes, sección 8.2.5.16): leer 6 registros desde el 3 y escribir 0x00FF en los registros 14, 15 y 16. Los registros 3 a 8 valen 0x00FE, 0x0ACD, 1, 3, 0x000D y 0x00FF.

Petición

	Cabecera MBAP	Función	Dir. lectura	Cant. lectura	Dir. escritura	Cant. escritura	Número de bytes N	Valores a escribir
Bytes	7	1	2	2	2	2	1	N
Valores	Length = 11 + N	`17`	0–65535	1–125	0–65535	1–121	2 × cant. escritura	2 bytes por registro
Ejemplo	`00 0A 00 00 00 11 01`	`17`	`00 03` = 3	`00 06` = 6	`00 0E` = 14	`00 03` = 3	`06`	`00 FF` `00 FF` `00 FF`

Respuesta

	Cabecera MBAP	Función	Número de bytes N	Valores leídos
Bytes	7	1	1	N
Valores	Length = 3 + N	`17`	2 × cant. lectura	2 bytes por registro
Ejemplo	`00 0A 00 00 00 0F 01`	`17`	`0C` = 12	`00 FE` `0A CD` `00 01` `00 03` `00 0D` `00 FF`

```
TX: 00 0A 00 00 00 11 01 17 00 03 00 06 00 0E 00 03 06 00 FF 00 FF 00 FF
RX: 00 0A 00 00 00 0F 01 17 0C 00 FE 0A CD 00 01 00 03 00 0D 00 FF
```

Respuesta de excepción

Cuando la cabecera es válida pero el servidor no puede atender la petición, responde con una PDU de 2 bytes: el código de función pedido con el bit 7 a 1 (función + 0x80) y un código de excepción.

Ejemplo de la tabla: leer el registro 10 000 (0x2710) de un servidor cuyas tablas acaban en 9999.

	Cabecera MBAP	Función + 0x80	Código de excepción
Bytes	7	1	1
Valores	Length = 3	`81`, `82`, `83`…	Ver la tabla siguiente
Ejemplo	`00 0B 00 00 00 03 01`	`83` = 03 + 80	`02` = ILLEGAL DATA ADDRESS

```
TX: 00 0B 00 00 00 06 01 03 27 10 00 01
RX: 00 0B 00 00 00 03 01 83 02
```

Código	Nombre	Cuándo aparece
0x01	ILLEGAL FUNCTION	El servidor no implementa o no permite esa función
0x02	ILLEGAL DATA ADDRESS	Dirección, o dirección + cantidad, fuera del mapa del servidor
0x03	ILLEGAL DATA VALUE	Un valor del campo de datos es inválido: cantidad 0 o excesiva, número de bytes incoherente, valor de coil distinto de 0xFF00/0x0000, longitud de PDU incorrecta
0x04	SERVER DEVICE FAILURE	Error irrecuperable al ejecutar la acción
0x05	ACKNOWLEDGE	Petición aceptada; tardará en completarse
0x06	SERVER DEVICE BUSY	El servidor está ocupado; reintentar más tarde
0x08	MEMORY PARITY ERROR	Error de paridad al leer un fichero (funciones 20/21)

Código	Nombre	Cuándo aparece
0x0A	GATEWAY PATH UNAVAILABLE	La pasarela no tiene ruta hacia el destino
0x0B	GATEWAY TARGET DEVICE FAILED TO RESPOND	La pasarela no obtuvo respuesta del Unit ID indicado

Orden de las comprobaciones en el servidor: primero la función (0x01), después la cantidad y la longitud (0x03), después la dirección (0x02), y por último la ejecución (0x04). Una petición con varios errores recibe la excepción de la primera comprobación que falla.

Excepción, timeout o desconexión. Recibir una excepción es una buena noticia para el diagnóstico: la red, TCP y la cabecera funcionan, y el problema está en la PDU. Un timeout apunta a la cabecera, a un servidor demasiado lento o a la red.

Desarrollo en el laboratorio

Se implementa por niveles y en el orden de las tablas. Después de cada función, comprobar su `pdu()` con la trama de ejemplo del guion y ejecutarla contra el simulador. Si el simulador responde con una excepción o no responde, mirar primero el analizador de tramas: indica qué campo está mal.

Nivel básico (para aprobar)

Orden	Función	Clase y atajo	Pista
1	Base del cliente y 03 Read Holding Registers	`CabeceraMBAP`, `ClienteModbusTCP` (`conectar`, `cerrar`, `ejecutar`), `LeerHoldingRegisters`, `leer_holding_registers`	Empezar aquí: con la 03 funcionando, el resto reutiliza el mismo cliente
2	04 Read Input Registers	`LeerInputRegisters`, `leer_input_registers`	Igual que 03: conviene una clase intermedia común
3	01 Read Coils	`LeerCoils`, `leer_coils`	Comprobar el número de bytes y desempaquetar los bits
4	02 Read Discrete Inputs	`LeerEntradasDiscretas`, `leer_entradas_discretas`	Igual que 01
5	06 Write Single Register	`EscribirRegistro`, `escribir_registro`	Los negativos se envían en complemento a 2. Comprobar el eco
6	05 Write Single Coil	`EscribirCoil`, `escribir_coil`	ON no es 1
7	Errores	`ExcepcionModbus`, `ErrorTimeout`, `ErrorConexion`	Ver abajo cómo provocar cada uno

Nivel avanzado (para optar a la máxima nota)

Solo se evalúa si el nivel básico está completo.

Orden	Función	Clase y atajo	Pista
8	10 Write Multiple Registers	`EscribirRegistros`, `escribir_registros`	El número de bytes va entre la cantidad y los valores
9	0F Write Multiple Coils	`EscribirCoils`, `escribir_coils`	Empaquetar los bits como en la función 01
10	17 Read/Write Multiple Registers	`LeerEscribirRegistros`, `leer_escribir_registros`	El servidor escribe antes de leer
11	Programa de línea de órdenes	`modbus_cli.py`	Ver abajo

Programa `modbus_cli.py`. Ejecuta una sola petición con la biblioteca propia: `python3 modbus_cli.py [--ip IP] [--puerto P] [--unidad U] [--timeout S] [--traza] ORDEN ARGUMENTOS`. Las órdenes son `leer-coils`, `leer-entradas`, `leer-holding` y `leer-input` (dirección y cantidad), `escribir-coil` y `escribir-registro` (dirección y valor), `escribir-coils` y `escribir-registros` (dirección y valores) y `leer-escribir` (dirección de lectura, cantidad, dirección de escritura y valores). La salida es exacta:

Caso	Salida	Código de salida
Lectura de bits	Una línea por bit: `direccion valor`	0
Lectura de registros	Una línea por registro: `direccion sin_signo con_signo 0xHHHH`	0
Escritura correcta	`OK`	0
Argumento fuera de rango	`ARGUMENTO INVALIDO: <motivo>`	2
Excepción Modbus	`EXCEPCION 0xCC NOMBRE`	3
Timeout	`TIMEOUT`	4
Error de conexión	`ERROR DE CONEXION: <motivo>`	5
Error de protocolo	`ERROR DE PROTOCOLO: <motivo>`	6

Cómo provocar cada error contra el simulador:

- `ExcepcionModbus`: leer registros a partir de la dirección 9999, o deshabilitar una función en *Comportamiento*. Después, el cliente debe seguir funcionando.
- `ErrorTimeout`: en *Comportamiento*, atender solo al Unit ID 1 con la política *Descartar*, y hacer una petición con otro Unit ID.
- `ErrorConexion`: conectar a un puerto en el que no escucha nadie.

Cierre

Demostrar al profesor, desde la consola de Python, una lectura, una escritura y una excepción con la biblioteca propia. El programa `modbus_cli.py` y las funciones 0F, 10 y 17 forman parte de la entrega solo para quien opte al nivel avanzado.

Entregables y evaluación

La entrega es individual:

- Carpeta `practica1` con `modbus_tcp.py` (con la variable `AUTOR`) y `modbus_cli.py`.

La nota se obtiene por niveles:

Nivel	Qué se evalúa	Cómo	Nota
Básico	MBAP, funciones 01–06 y errores	Corrector automático contra el simulador	7 si se superan todas las pruebas básicas
Avanzado	Funciones 0F, 10 y 17	Corrector automático	hasta +2
Calidad	Reutilización mediante clases, documentación, legibilidad	Revisión del profesor	hasta +1

El código debe ser propio: el profesor puede pedir que se explique cualquier parte.